

```
section .data
    input_prompt db "Enter the 5-digit BCD number: ", 0
    input_prompt_len equ $ - input_prompt
    hex_result_msg db "Hexadecimal equivalent: ", 0
    hex_result_msg_len equ $ - hex_result_msg
```

```
section .bss
    bcd_number resb 6
    hex_result resb 4
    intermediate_result resw 1
```

```
%macro print_string 2
    mov rax, 1
    mov rdi, 1
    mov rsi, %1
    mov rdx, %2
    syscall
%endmacro
```

```
%macro read_input 2
    mov rax, 0
    mov rdi, 0
    mov rsi, %1
    mov rdx, %2
    syscall
%endmacro
```

```
%macro exit_program 0
    mov rax, 60
    xor edi, edi
    syscall
%endmacro
```

```
section .text
    global _start
```

```
_start:
    print_string input_prompt, input_prompt_len
    call convert_BCD_to_HEX
    exit_program
```

```
display_hex_result:
    mov rbx, 16
    mov rcx, 4
    mov rsi, hex_result + 3
convert_to_hex_loop:
    xor rdx, rdx
    div rbx
    cmp dl, 9
    jbe digit
    add dl, 'A' - 10
    jmp next
```

```
digit:
    add dl, '0'
next:
    mov [rsi], dl
    dec rsi
    dec rcx
    jnz convert_to_hex_loop
    print_string hex_result_msg, hex_result_msg_len
    print_string hex_result, 4
    ret
```

```
convert_BCD_to_HEX:
    read_input bcd_number, 6
    mov rsi, bcd_number
    xor rax, rax
    mov rbx, 10
    mov rcx, 5
convert_loop:
    xor rdx, rdx
    mul rbx
    mov dl, [rsi]
    sub dl, '0'
    add rax, rdx
    inc rsi
    dec rcx
    jnz convert_loop
    mov [intermediate_result], rax
    call display_hex_result
    ret
```